Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 04 Notes
Control Flow + Arrays + Strings

Tofik Ali

February 26, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 01) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
- 5.1 Program control statements (selection + loops) . . . 2
  - 5.1.1 `if/else` . . . 2
  - 5.1.2 `switch` . . . 2
  - 5.1.3 Loops . . . 3
- 5.2 Arrays (1D/2D), types of arrays, common patterns . . . 3
  - 5.2.1 1D arrays . . . 3
  - 5.2.2 2D and jagged arrays . . . 3
- 5.3 Strings and `StringBuilder` . . . 3
  - 5.3.1 String immutability . . . 3
  - 5.3.2 `==` vs `equals` . . . 4
  - 5.3.3 When to use `StringBuilder` . . . 4

6 Worked Example: Grades (control flow + arrays + strings) **4**

7 In-Class Exercises (with brief solutions) **5**

8 Self-Study Practice (no solutions) **5**

9 Checkpoint Questions (with sample answers) **5**

1 Lecture Overview

This lecture teaches you how to **control program flow** (decisions + loops) and work with two core data structures: **arrays** (for many values of the same type) and **strings** (text).

These three topics appear in almost every lab/program you will write.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

2 Syllabus Alignment (Unit 01)

Unit 01: Control statements, arrays, and strings

3 Learning Outcomes

After this lecture, you should be able to:

- Use selection statements (`if/else`, `switch`) and loops (`for/while/do-while`).
- Create and traverse 1D/2D arrays and solve common array problems (sum, max, search).
- Explain why `String` is immutable and when to use `StringBuilder`.

4 Key Terms (Quick)

- **Selection**: choosing one path among many (`if`, `switch`).
- **Iteration**: repeating work until a condition changes (`for`, `while`).
- **Array**: fixed-size contiguous collection of same-type elements.
- **Index**: position in array (0-based in Java).
- **Immutable**: cannot be changed after creation (e.g., `String`).
- **StringBuilder**: mutable sequence of characters (efficient for repeated concatenation).

5 Detailed Notes

5.1 Program control statements (selection + loops)

5.1.1 `if/else`

Use `if` when you have conditions based on boolean expressions.

Good practice:

- keep conditions readable (break complex conditions into variables),
- avoid deeply nested `if` blocks (use early returns in methods later).

5.1.2 `switch`

Use `switch` when one variable is compared against many constant values. It often makes menu-driven programs readable.

Pitfall: forgetting `break` can cause fall-through (sometimes intended, often a bug).

5.1.3 Loops

- `for`: when you know the number of iterations (or array traversal).
- `while`: when you loop until a condition changes.
- `do-while`: when the body must run at least once.

Pitfall: off-by-one errors. Remember arrays are 0-based and last index is `length - 1`.

5.2 Arrays (1D/2D), types of arrays, common patterns

5.2.1 1D arrays

Create:

- `int[] a = new int[5];` (all zeros initially)
- `int[] b = {1,2,3};` (initialized)

Access:

- `a[i]` for element, `a.length` for size (not a method).

Common patterns:

- sum/average, max/min, linear search, counting, reversing.

5.2.2 2D and jagged arrays

2D array:

- `int[][] m = new int[rows][cols];`

Jagged array (rows can have different lengths):

- `int[][] j = new int[3][];` then allocate each row separately.

Pitfall: always check `m[i].length` for column length in jagged arrays.

5.3 Strings and `StringBuilder`

5.3.1 String immutability

In Java, `String` is immutable: operations like `toLowerCase()` or `trim()` return a new string.

Why it matters:

- Safer sharing: if two variables reference the same string, it can't be modified unexpectedly.
- Performance implication: repeated concatenation in loops creates many temporary objects.

5.3.2 `==` vs `equals`

Use:

- `==` to compare references (same object or not),
- `equals()` to compare string content.

5.3.3 When to use `StringBuilder`

If you build a string repeatedly (especially in a loop), use `StringBuilder` and `append()`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

6 Worked Example: Grades (control flow + arrays + strings)

```java
import java.util.Scanner;

public class GradeStats {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        int[] marks = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Marks for student " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }

        int max = marks[0];
        int sum = 0;
        for (int x : marks) {
            sum += x;
            if (x > max) max = x;
        }

        double avg = sum / (double) n;
        System.out.printf("Max=%d, Avg=%.2f%n", max, avg);

        String result = (avg >= 40) ? "PASS" : "NEEDS IMPROVEMENT";
        System.out.println("Class status: " + result);

        sc.close();
    }
}
```

What to notice:

- Off-by-one handling in prompts: `i+1` for human-friendly numbering.

- Enhanced for-loop (`for (int x : marks)`) to traverse arrays.
- Casting for average: `sum / (double)n` to avoid integer division.

7 In-Class Exercises (with brief solutions)

1. What is the last valid index of an array of length 10? **Answer:** 9.
2. Should we use `==` or `equals` to compare strings by content? **Answer:** `equals`.
3. Which loop runs at least once: `while` or `do-while`? **Answer:** `do-while`.

8 Self-Study Practice (no solutions)

1. Write a program to find the second largest element in an array.
2. Write a program to check if a string is a palindrome (ignore spaces and case).
3. Create a 2D matrix and compute row-wise sums.

9 Checkpoint Questions (with sample answers)

Q1. Why is `String` immutable and why does it matter?

Sample answer: Immutability makes strings safe to share and helps with security and caching. It also means operations create new objects, so repeated concatenation in loops can be inefficient.

Q2. How do you avoid off-by-one errors in array loops?

Sample answer: Remember Java arrays are 0-based and the last index is `length - 1`. Use loop conditions like `i < a.length` rather than `i <= a.length`.